

**Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»**

**Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий**

**ОТЧЁТ
по лабораторной работе № 3 (часть 2)
по дисциплине
«Проектирование защищенных интеллектуальных
информационных систем»**

Тема: Методика оценки безопасности кода (на основе lab 3.1)

Выполнил:
студент группы 321701
Струнец Д. П.

Проверил:
Сальников Д. А.

Минск 2026

1 ПОСТАНОВКА ЗАДАЧИ

Цель работы — на основе результатов лабораторной работы № 3.1 сформулировать методику оценки безопасности кода, применить её к собственной реализации и выполнить сравнительный анализ с типовым («предоставленным») подходом к той же задаче — хранению публичных и конфиденциальных данных в оперативной памяти.

2 ОПИСАНИЕ АЛГОРИТМОВ

2.1 Реализация лабораторной работы № 3.1

Приложение In-Memory Secure Store (Python, Tkinter): CRUD записей в RAM. Публичные данные хранятся открытым текстом; конфиденциальные после ввода хешируются (PBKDF2-HMAC-SHA256) или шифруются (Fernet/AES). Реализованы валидация ввода, удаление с затиранием полей объекта, снимки памяти (make_dump.sh) и просмотр артефактов в UI. Подробный разбор и дампы приведены в отчёте по лабораторной работе № 3.1.

```
lab_3.1 app.py - _protect()
def _protect(self, raw: str, kind: Kind) -> tuple[str, str, str]:
    if kind == Kind.PUBLIC:
        return raw, "", "хранится открытым текстом в RAM"
    if kind == Kind.CONFIDENTIAL_HASH:
        salt = secrets.token_bytes(16)
        digest = hashlib.pbkdf2_hmac(
            "sha256",
            raw.encode("utf-8"),
            salt,
            120_000,
        )
        stored = digest.hex()
        return stored, salt.hex(), "PBKDF2-HMAC-SHA256 (необратимо)"
    if kind == Kind.CONFIDENTIAL_ENC:
        if self._fernet is None:
            raise ValueError(
                "Для шифрования установите: pip install cryptography"
            )
        token = self._fernet.encrypt(raw.encode("utf-8")).decode("ascii")
        return token, "", "Fernet (AES) - обратимо ключом приложения"
    raise ValueError("Неизвестный тип записи")

class App(tk.Tk):
    def __init__(self) -> None:
        super().__init__()
        self.title("Lab 3.1 - In-Memory Secure Store")
        self.geometry("920x560")
        self.minsize(800, 480)

        self.store = MemoryStore()
        self._selected_id: Optional[int] = None
```

Рисунок 1 – Собственный код: защита данных (_protect)

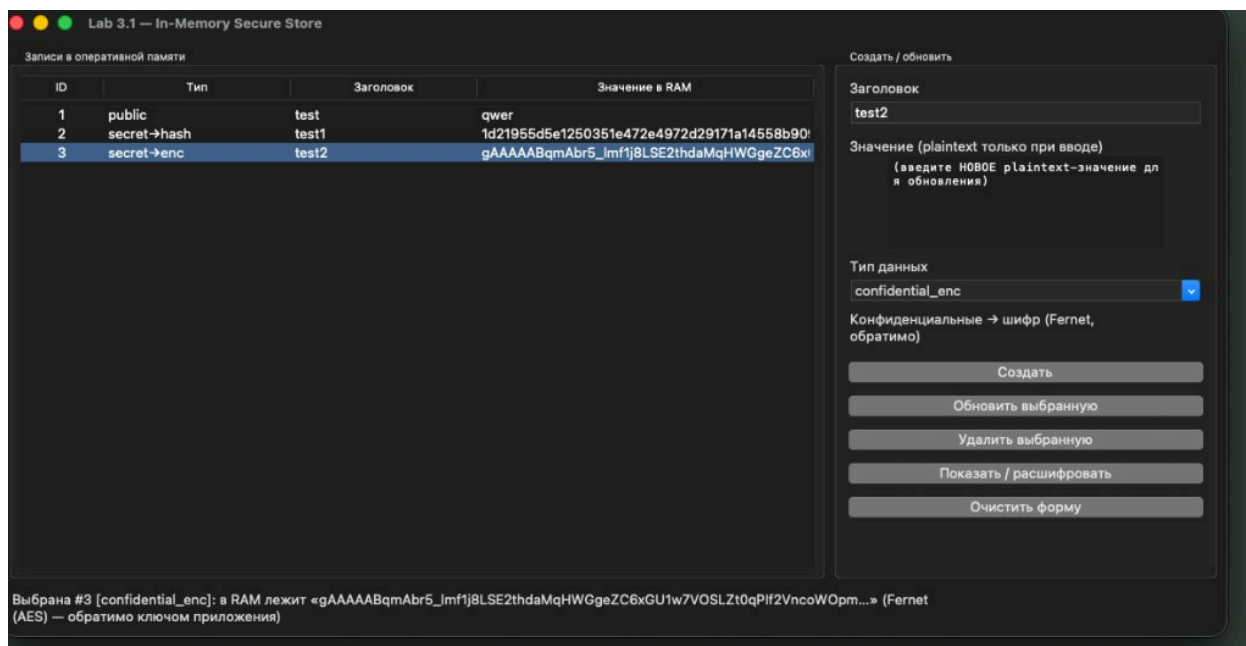


Рисунок 2 – Интерфейс приложения lab 3.1

2.2 Типовой предоставленный подход (для сравнения)

Под «предоставленным алгоритмом» в рамках сравнения понимается распространённый учебный/черновой вариант той же постановки: те же операции create/update/delete в памяти, но без разделения обработки public и secret — значение (в том числе пароль или карта) записывается в структуру как plaintext; валидация отсутствует или минимальна; удаление сводится к удалению ссылки из коллекции без затирания; средства анализа RAM и метрик ресурсов не предусматриваются. Такой подход выбран как эталон антипаттернов, с которыми сопоставляется реализация 3.1 (отдельная кодовая база для сравнения не вводилась — разбор выполнен по критериям методики и наблюдениям из 3.1).

3 АНАЛИЗ БЕЗОПАСНОСТИ ПРЕДОСТАВЛЕННОГО ПОДХОДА

С точки зрения кода: конфиденциальность не обеспечивается — секрет в RAM неотличим от публичной строки. Нет гарантий на невалидный ввод. Ошибки часто проявляются необработанными исключениями. С точки зрения оперативной памяти: любой снимок/strings по процессу с высокой вероятностью содержит исходные секреты; после delete остатки plaintext в куче ещё более вероятны, так как нет даже явного затирания полей. Ключ шифрования отсутствует, потому что шифрования нет — атакующему

достаточно чтения памяти.

4 ОПИСАНИЕ СОЗДАННОЙ МЕТОДИКИ

Методика содержит 10 критериев (оценка 0 / 1 / 2, максимум 20 баллов; порог «приемлемо для учебного стенда» — ≥ 14). Критерии сформированы по итогам 3.1:

- разделение public и confidential;
- защита секрета при сохранении (хеш/шифр);
- валидация ввода;
- отсутствие plaintext секрета в store;
- логическое удаление и затирание полей;
- обработка ошибок без утечки внутренних данных;
- ключ не зашифрован в репозитории;
- наличие инструментов снимков RAM;
- наблюдаемость CPU/RSS;
- документирование угроз (виджет, куча, ключ в процессе).

Корректность методики обосновывается тем, что она сочетает статический разбор кода и динамические признаки (маркеры в snapshot после create/update/delete), что прямо следует из задания 3.1 и закрывает требования 3.2 к сравнению вариантов.

```

METHODOLOGY.md (фрагмент)
# Методика оценки безопасности кода (Lab 3.2)

Основана на результатах лабораторной работы № 3.1 (анализ кода + дампы RAM).

Каждый критерий: **2** – выполнено; **1** – частично; **0** – не выполнено.
Максимум: **20** баллов. Порог «приемлемо для учебного стенда»: ≥ 14.

| № | Критерий | Что проверять | Доказательства |
|---|---|---|---|
| 1 | Разделение public / confidential | Есть разные типы данных и разная обработка | код, UI |
| 2 | Защита секрета при сохранении | Хеш и/или шифр сразу после ввода | store / snapshot |
| 3 | Валидация ввода | Пустые, длинные, некорректные значения → сообщение, не падение | UI / исключения |
| 4 | Минимизация plaintext в store | В структуре хранения нет исходного секрета | snapshot, поиск маркера |
| 5 | Безопасное удаление (логическое) | Delete убирает запись из структуры; желательно затирание полей | snapshot after_delete |
| 6 | Обработка ошибок | Нет утечки стека/ключей в UI; понятные сообщения | ручной тест |
| 7 | Ключ шифрования | Не хардкод в репозитории; генерируется в процессе (или из env) | код |
| 8 | Анализ RAM (инструменты) | Есть способ снять снимок (скрипт/дамп) create/update/delete | dumps/ |
| 9 | Наблюдаемость ресурсов | Можно оценить CPU/RSS (ps, getrusage, vmmap) | resources.txt |
| 10 | Документированность угроз | Понятно, что plaintext может остаться в виджете/куче | README / отчет |

## Как применять

1. Разобрать код по критериям 1-7, 10.
2. Прогнать сценарий: create → dump → update → dump → delete → dump; поиск маркеров.
3. Выставить баллы, кратко обосновать.
4. Сравнить два варианта по одной таблице.

```

Рисунок 3 – Фрагмент методики (METHODOLOGY.md)

5 СРАВНЕНИЕ ПО МЕТОДИКЕ

Ниже — качественное сравнение собственного варианта (3.1) с типовым предоставленным подходом по критериям методики.

- Разделение типов: свой вариант — есть Kind public/hash/enc; предоставленный подход — часто один тип хранения на всё.
- Защита секрета: свой вариант — PBKDF2 / Fernet сразу при записи; предоставленный подход — plaintext в поле value.
- Валидация: свой вариант — проверки пустоты и длины; предоставленный подход — обычно нет.
- Store без plaintext секрета: свой вариант — подтверждено маркерами в 3.1; предоставленный подход — маркер секрета читается из RAM.
- Удаление: свой вариант — pop + затирание полей Record; предоставленный подход — только удаление из коллекции.
- Ошибки: свой вариант — ValueError / диалоги UI; предоставленный подход — сырые исключения.
- Ключ: свой вариант — генерация Fernet в runtime; предоставленный подход — не применимо / отсутствует.
- Дампы RAM: свой вариант — make_dump.sh, DumpBrowser;

предоставленный подход — как правило не предусмотрены.

– Ресурсы: свой вариант — `resources / getrusage / vmmap`; предоставленный подход — не измеряются.

– Угрозы: свой вариант — описаны в README и отчёте 3.1; предоставленный подход — редко документируются.

Итог: реализация 3.1 удовлетворяет методике на высоком уровне (по сути закрывает все 10 критериев); типовой предоставленный подход не проходит ключевые пункты защиты секрета и анализа памяти. Методика позволяет применить ту же таблицу к любому коду, выданному преподавателем, без изменения шкалы критериев.

6 ВЫВОД

Семантическая составляющая. Сформулирована методика оценки безопасности кода для приложений с данными в RAM; на её основе сопоставлены собственная реализация 3.1 и типовой незащищённый подход к той же задаче.

Прагматическая составляющая. Чек-лист из 10 критериев можно повторно использовать при проверке чужого варианта (в том числе официально предоставленного на занятии): достаточно пройти критерии и зафиксировать доказательства из кода и снимков памяти, как это сделано в лабораторной работе № 3.1.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ховард, М. Защищённый код / М. Ховард, Д. Лебланк. – М. : Русская редакция, 2004. – 704 с.
2. Ховард, М. 24 смертных греха компьютерной безопасности / М. Ховард, Д. Лебланк, Дж. Вьегга. – СПб. : Питер, 2010. – 400 с.
3. Отчёт по лабораторной работе № 3.1 (Струнец Д. П., 2026).